Deep Learning Compilation for the NVIDIA H100: Lowering a Tiny MLP with MLIR and torch-mlir

C. Alejandra Carreón-Jiménez

December 2025

Abstract

This report describes my approach for the "Deep Learning Compilation for the NVIDIA H100 GPU" assignment. The goal was to start from a PyTorch deep learning model and successively lower it through several intermediate representations (IRs) down to low-level GPU-targeted code using the MLIR and torch-mlir toolchains. I chose to work with the provided tiny MLP example and focused on producing a complete lowering pipeline from the PyTorch model, through the MLIR Torch and Linalg dialects, down to GPU/NVVM/LLVM IR specialized for the NVIDIA H100 (sm_90). I document the environment, the compilation flow, what worked, what did not, and possible next steps.

1 Introduction

The objective of this assignment is to experiment with deep learning compilation techniques targeting an NVIDIA H100 GPU (or a MIG instance of an H100). Instead of treating PyTorch as a black box, the idea is to explicitly lower a model through multiple intermediate representations and observe how high-level tensor operations eventually become low-level GPU kernels.

I chose to use the tiny MLP model provided by the instructor as the starting point. The compilation flow I implemented is inspired by Stephen Diehl's MLIR GPU tutorial[1], but adapted to start from the MLIR Torch dialect produced by torch-mlir instead of a hand-written Linalg kernel. The main steps are:

- Export the PyTorch MLP to MLIR (Torch dialect) using torch-mlir.
- Lower Torch → Linalg on tensors using `torch-mlir-opt`.
- Lower Linalg → Affine → GPU → NVVM → LLVM IR targeting sm_90 using a custom MLIR pass pipeline.

The result is a GPU-ready MLIR module, `tiny_mlp_gpu.mlir`, which contains NVVM/LLVM IR suitable for generating PTX for the H100 architecture. Due to time and scope constraints, I did not fully integrate the final PTX generation and runtime execution, but I outline how this could be done as future work.

[1] https://www.stephendiehl.com/posts/mlir_gpu/

2 Environment and Toolchain

2.1 Hardware and Cluster Setup

All experiments were run on our campus cluster (Punakha) using a node with an NVIDIA H100 HGX GPU. The H100 is MIG'd into smaller GPU instances, each with 10 GB of memory and 16 SMs, but still exposing the Hopper architecture (sm_90). The assignment setup script and instructions from the instructor were used as the starting point.

2.2 Software Environment

Instead of building LLVM/MLIR completely from source (which would be very time-consuming with limited cores), I decided to use the MLIR toolchain already available inside Stephen Diehl's Docker container. On the node, the environment was roughly:

- Base container: Stephen Diehl's MLIR GPU Docker image (as referenced in the assignment).
- MLIR tools:
  - `mlir-opt` version 20.1.5 (from the container).
- PyTorch stack:
  - PyTorch 2.9.1 with CUDA 12.8 wheels (`cu128` index).
  - `torchvision`, `torchaudio` installed via the same index.
- Torch-MLIR:
  - Installed from the official dev wheels with:

```
pip install --break-system-packages \
  --force-reinstall \
  --target=/root/.local/lib/python3.12/site-packages \
  -f https://github.com/llvm/torch-mlir-release/releases/expanded_assets/dev-
    wheels \
  torch_mlir
```

  - `torch-mlir-opt` available on the path.
  - A minor warning about `__version__` not being set was observed, but the toolchain worked correctly.

The decision to reuse the container's MLIR (v20.1.5) and install a compatible torch-mlir dev wheel is important: the assignment notes warn about potential version mismatches between MLIR and torch-mlir. I chose the pragmatic route of making the existing container work rather than recompiling LLVM/MLIR from scratch.

3 Model: Tiny MLP in PyTorch

For the model, I used the tiny MLP script provided by the instructor, `tiny_mlp.py`. Conceptually, the model is a small fully-connected network (multi-layer perceptron) with two linear layers and a nonlinearity, e.g.:

- Input: vector of fixed dimension (e.g., 4 or 10).
- Hidden layer: `nn.Linear(in_dim, hidden_dim)` + `ReLU`.
- Output layer: `nn.Linear(hidden_dim, out_dim)`.

A typical structure looks like:

```
import torch
import torch.nn as nn
import torch_mlir

class TinyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(4, 8)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(8, 4)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

The `tiny_mlp.py` script also:

- Forces execution on CPU (to avoid CUDA device detection issues during export).
- Uses `torch-mlir` to compile the model to the MLIR Torch dialect.
- Writes the resulting IR to `tiny_mlp.mlir`.

4 Lowering Torch to Linalg

4.1 Export to MLIR Torch Dialect

The first step is to export the PyTorch model to the MLIR Torch dialect. Running:

```
python3 tiny_mlp.py
```

produces:

- `tiny_mlp.mlir` (Torch dialect).

This file represents the model in terms of the torch-mlir operations, still relatively high-level and close to PyTorch semantics. At this stage, the representation is mostly hardware agnostic.

4.2 Torch → Linalg

Next, I lowered the model from the Torch dialect to Linalg on tensors using `torch-mlir-opt`:

```
torch-mlir-opt tiny_mlp.mlir \
  --torch-decompose-complex-ops \
  --torch-backend-to-linalg-on-tensors-backend-pipeline \
  -o tiny_mlp_linalg.mlir
```

The most important aspects of this step are:

- **Decomposition**: complex Torch ops are decomposed into simpler pieces.
- **Linalg backend pipeline**: converts the Torch-level representation into Linalg ops on tensors, which are much closer to explicit linear algebra kernels.

The output, `tiny_mlp_linalg.mlir`, is now expressed largely in the Linalg dialect, ready to be lowered to loops and then GPU kernels.

5 Lowering Linalg to GPU/NVVM/LLVM

5.1 Custom Lowering Script

For the second part of the assignment, I wrote a Python script, `mlp_to_gpu.py`, to orchestrate the lowering from Linalg down to GPU/NVVM/LLVM. The script:

- Detects the GPU target (sm_90 for NVIDIA H100).
- Builds a pass pipeline using `mlir-opt` to progressively lower the IR.
- Writes a log to `result.txt` and the final IR to `tiny_mlp_gpu.mlir`.

Conceptually, the pass pipeline in the script is:

1. `canonicalize` – simplify IR.
2. `one-shot-bufferize` – convert tensors to memrefs.
3. `convert-linalg-to-affine-loops` – Linalg → Affine loops.
4. `affine-loop-invariant-code-motion` – loop optimizations.
5. `convert-affine-for-to-gpu` – Affine loops → GPU kernels.
6. `gpu-kernel-outlining` – extract GPU kernels into a separate module.
7. `lower-affine` – lower remaining affine constructs to standard/LLVM-compatible ops.
8. `gpu-decompose-memrefs` – prepare memory ops for GPU.
9. `convert-gpu-to-nvvm` – GPU dialect → NVVM dialect (NVIDIA-specific).
10. `nvvm-attach-target` – attach target attributes for sm_90 / PTX 8.0.
11. `convert-nvvm-to-llvm` – NVVM → LLVM dialect.
12. `gpu-to-llvm` – finalize GPU-related lowering.

Running:

```
python3 mlp_to_gpu.py
```

applies all of these passes to `tiny_mlp_linalg.mlir` and produces:

- `tiny_mlp_gpu.mlir` (final GPU/NVVM/LLVM-level IR).
- `result.txt` (a human-readable summary of the pipeline and statistics).

5.2 Summary of `result.txt`

The script prints a structured log; a shortened version is:

```
================================================================================
Step 3: Lowering MLP from Linalg to GPU/NVVM/PTX
================================================================================

Detected GPU architecture: sm_90
 Target: NVIDIA H100 (Hopper architecture)

Reading input file: tiny_mlp_linalg.mlir
 Loaded 3,116 bytes

... (pass pipeline summary) ...

 GPU-ready MLIR saved to: tiny_mlp_gpu.mlir
 File size: 29,647 bytes

MLIR statistics:
  Total lines: 464
  GPU kernel references: 9
  NVVM operations: 36
  PTX indicators: 27
```

This confirms that the lowering pipeline successfully produced a GPU-oriented module with multiple GPU kernel references and a non-trivial number of NVVM operations.

5.3 Structure of the Final IR

The beginning of `tiny_mlp_gpu.mlir` looks like:

```
module attributes {gpu.container_module} {
  llvm.func @malloc(i64) -> !llvm.ptr
  llvm.mlir.global private constant @__constant_2xf32(
      dense_resource<torch_tensor_2_torch.float32> : tensor<2xf32>)
      {addr_space = 0 : i32, alignment = 64 : i64}
      : !llvm.array<2 x f32>
  ...
  llvm.func @main(%arg0: !llvm.ptr) -> !llvm.ptr {
    %0 = llvm.mlir.poison :
         !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2 x i64>)>
    ...
```

Later in the file, GPU kernels and NVVM intrinsics appear, reflecting that the Linalg-level operations have been fully lowered into Hopper-targeted NVVM/LLVM IR.

6 Challenges and Limitations

The main challenges and limitations in this assignment were:

6.1 Version Compatibility

The assignment notes warn that the MLIR version in Diehl's container (v20) and the torch-mlir version to be installed (v22) may be incompatible. Initially, I considered recompiling LLVM/MLIR v22 from source, but this would have taken several hours of compilation time on the available hardware.

Instead, I:

- Reused the MLIR tools (`mlir-opt`, etc.) that come with the container.
- Installed a recent torch-mlir dev wheel and verified that:
  - `torch-mlir-opt` functioned correctly.
  - The basic pipelines (Torch → Linalg) worked without crashes.

This pragmatic choice allowed me to focus on the lowering pipeline and analysis, while still acknowledging the potential fragility of mixing tool versions.

6.2 Execution and PTX Generation

The tutorial by Diehl lowers a hand-written square kernel all the way to PTX and then executes it via the CUDA driver API. In my case:

- I successfully lowered the tiny MLP all the way to GPU/NVVM/LLVM IR oriented to sm_90.
- However, I did not complete the final steps of:
  1. Translating the LLVM dialect module to textual LLVM IR (e.g., via `mlir-translate`).
  2. Using `clang` or similar to generate PTX.
  3. Writing a host-side driver (in Python or C++) to load the PTX, allocate buffers, and execute the kernel.

These steps are feasible -especially by adapting the `square2ptx.py` example from the tutorial- but were out of scope given the time and complexity constraints. Instead, I focus on documenting the lowering and the structure of the resulting GPU/NVVM/LLVM IR.

7 Conclusions and Future Work

In this assignment, I implemented an end-to-end lowering pipeline for a tiny MLP model:

- PyTorch model (tiny MLP) → MLIR Torch dialect via torch-mlir.
- Torch dialect → Linalg on tensors via `torch-mlir-opt`.
- Linalg → Affine → GPU → NVVM → LLVM dialect using a custom pass pipeline in `mlp_to_gpu.py`, targeting sm_90 (NVIDIA H100).

The final artifact, `tiny_mlp_gpu.mlir`, is a GPU-ready module that encodes the model as NVVM/LLVM IR with multiple GPU kernel references. The `result.txt` summary confirms that the pipeline applied successfully and produced non-trivial GPU-oriented IR.

As future work, I would like to:

- Integrate the last steps of the compilation pipeline:
  - LLVM dialect → textual LLVM IR (`.ll`) using `mlir-translate`.
  - LLVM IR → PTX using `clang` targeting `nvptx64-nvidia-cuda` and `sm_90`.
  - Runtime execution of the compiled MLP on the H100 using the CUDA driver API (or Python bindings).
- Experiment with more complex models, such as small CNNs or Transformers, to see how the structure of the IR scales with model complexity.
- Compare this MLIR/torch-mlir based approach with other deep learning compilers such as IREE or TVM, focusing on ease of use, performance, and transparency of the generated IR.

Overall, this project gave me hands-on experience with the full stack of deep learning compilation for GPUs, from PyTorch models all the way down to GPU-specialized IR, and a better intuition for how high-level tensor operations are transformed into low-level GPU kernels.